Elasticsearch Architecture

shards for size, replicas for safety and reads

Watch the 4-min video

In one line: an Elasticsearch/OpenSearch **cluster** stores each **index** as several **shards** (each a full Lucene index) spread over nodes, keeps **replica** copies for failover and read throughput, makes new documents searchable after a **refresh**, and stays healthy only with explicit mappings and sensibly sized shards.

1 The one-node trap

The team ships logs to Elasticsearch on one node with one big index. A year later a single shard holds 2 TB, searches time out, recovery after a restart takes hours, and losing the node means losing search. Elasticsearch and OpenSearch (a fork of Elasticsearch 7.10 made in 2021; same core architecture) scale horizontally, but only if indices, shards and nodes are planned.

2 Cluster, index, shard, replica

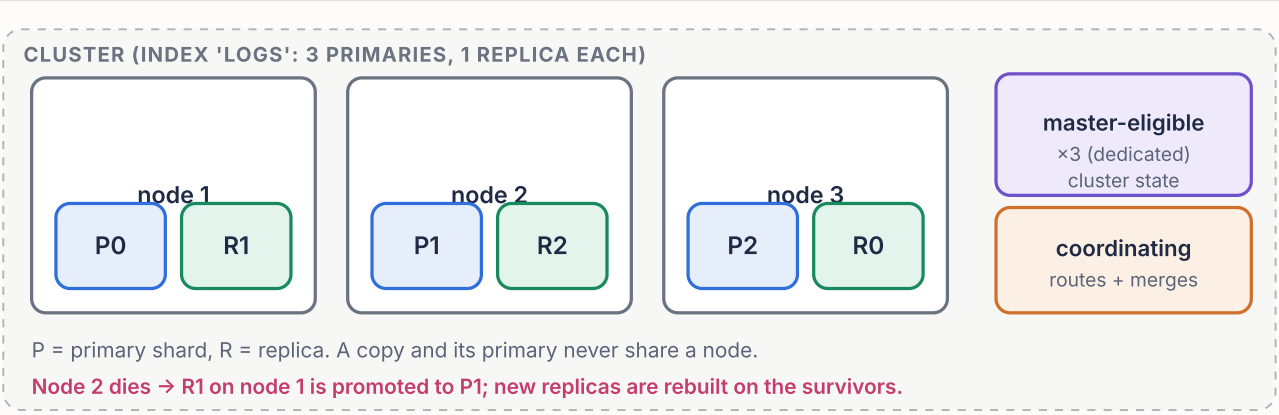


Figure 1. One index split into three primary shards, each with one replica on a different node.

- **Index**: a logical collection of documents with one mapping, roughly a table.
- **Shard**: one partition of an index and a complete Lucene index (lesson 15). A document goes to shard `hash(_routing) % number_of_primary_shards`, where `_routing` defaults to the document `_id`. Because the formula uses the primary count, that count is fixed at creation; changing it means `_split` / `_shrink` (by factors only) or a reindex.
- **Replica**: a full copy of a shard on another node. It gives failover (a replica is promoted if the primary's node dies) and read capacity (searches can use any copy). Replicas can be changed at any time. A single-node cluster can't place replicas, so health stays **yellow**.
- **Health**: green = all copies assigned; yellow = all primaries but some replicas unassigned; red = some primary unassigned, so some data is unavailable.

3 Write path and near-real-time search

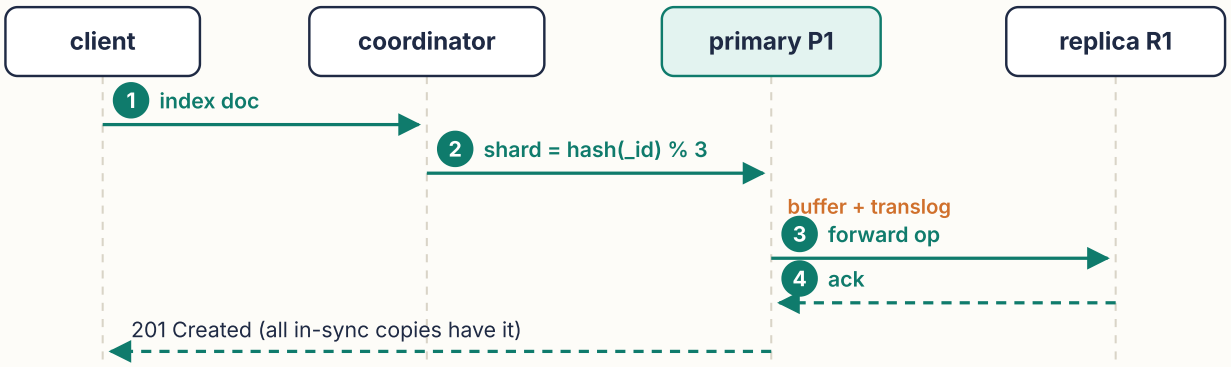


Figure 2. A write is routed to its primary, made durable in the translog, copied to every in-sync replica, then acknowledged.

The primary validates and indexes the document into its in-memory buffer and appends it to the **translog** (fsynced per request by default, so an acknowledged write survives a crash). It forwards the operation to the in-sync replicas in parallel and acknowledges when they have it. The document is **not yet searchable**: a **refresh** writes the buffer into a new segment, every `1s` by default, which is why search is called near real time (lesson 20). Elasticsearch also skips scheduled refreshes on shards that have not been searched for 30 s ('search idle'), so the first search after a quiet period may wait for one. For bulk loads, `refresh_interval: 30s` (or `-1` during a one-off load) makes indexing much cheaper: fewer, bigger segments and less merging.

4 Mappings: the schema you must own

A mapping says how each field is indexed. `text` is analysed for full-text search (lesson 14); `keyword` is one exact term, used for filters, sorting and aggregations (lesson 18); plus numbers, `date`, `ip`, `geo_point`, `dense_vector` (lesson 22). **Dynamic mapping** guesses types from the first document it sees, and by default maps strings as text with a keyword sub-field. In production that causes **mapping explosion**: logs that put user IDs or request headers into field names create thousands of fields, each costing cluster-state memory. The default `index.mapping.total_fields.limit` is 1,000. Set `dynamic: strict` (or `false`), and use the `flattened` type for arbitrary key-value bags. You can add new fields later, but changing an existing field's type or analyzer needs a reindex (lesson 19).

```
PUT _index_template/logs
{ "index_patterns": ["logs-*"],
  "template": { "settings": { "number_of_shards": 4, "number_of_replicas": 1,
      "refresh_interval": "30s", "index.lifecycle.name": "logs-30d" },
    "mappings": { "dynamic": "strict", "properties": {
      "@timestamp": { "type": "date" },
      "service":    { "type": "keyword" },
      "message":    { "type": "text" },
      "status":     { "type": "short" } } } } }
```

Figure 3. An index template: explicit mappings, primary/replica counts, a slower refresh for log ingestion and an ILM policy.

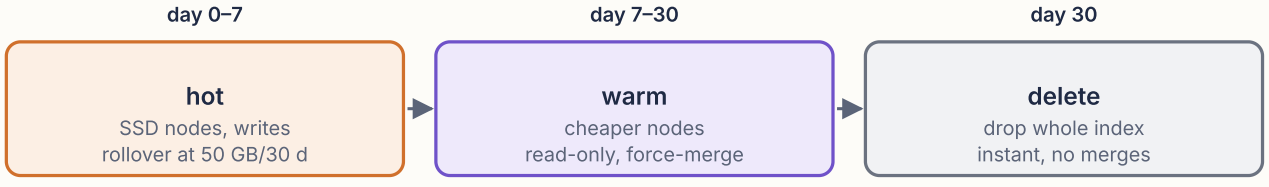
5 Sizing shards and nodes

Decision	Guideline	Why
Shard size	about 10–50 GB (Elastic's guidance); under ~200 M docs	bigger shards recover and rebalance slowly; tiny ones waste overhead
Shard count	avoid thousands of small shards per node	each shard costs heap, file handles, threads and cluster-state work
Primaries	enough to spread writes; fixed after creation	use rollover / time-based indices for growing data
Replicas	1 for safety; more only for read throughput	each replica re-indexes every write (lesson 25)
JVM heap	≤ 50% of RAM and below ~31 GB	the rest is OS page cache for segments; ~31 GB keeps compressed pointers
Master nodes	3 dedicated master-eligible nodes	a majority (2 of 3) is needed to elect a master; avoids split brain

Node roles separate work: master-eligible nodes keep the cluster state (indices, mappings, shard locations); data nodes hold shards and do the heavy indexing and searching, optionally tiered as `data_hot`, `data_warm`, `data_cold`; ingest nodes run pipelines; coordinating-only nodes just route and merge.

WORKED EXAMPLE: DAILY LOG INDICES WITH LIFECYCLE MANAGEMENT

Suppose services produce about 150 GB of logs a day. Each day gets a new index such as `logs-2026-10-31`, created from the template in Figure 3 with explicit mappings. Four primaries put each shard near 37 GB, inside the 10–50 GB band; with one replica that is 8 shards and 300 GB of disk per day. An ILM policy keeps an index on SSD **hot** nodes for 7 days, moves it to cheaper **warm** nodes (read-only, force-merged), and deletes it at 30 days. Steady state: 30 indices, 240 shards, about 9 TB including replicas. Deleting a day is **dropping a whole index**: instant, unlike a delete-by-query over millions of documents, which writes tombstones and triggers merges. Queries for 'the last hour' hit only today's index (lesson 25).



index template + ILM policy are applied automatically to every new logs- index (or data-stream backing index)*

Figure 4. Index lifecycle management (OpenSearch calls it Index State Management). Rollover can also cut indices by size instead of by day.

TRADE-OFFS

- **More primaries**: more write parallelism and smaller shards, but more per-query fan-out and overhead.
- **More replicas**: more read capacity and safety, but every write is done N+1 times and disk use multiplies.
- **Shorter refresh**: fresher results, but tiny segments and lower indexing throughput.
- **Strict mappings**: a stable cluster, but producers must coordinate schema changes.

COMMON MISTAKES

- Leaving dynamic mapping on for logs until a field explosion slows the whole cluster.
- One index per customer or per day at tiny volume: thousands of 50 MB shards ('oversharding').
- Only two master-eligible nodes: losing one leaves no majority, so the cluster can't elect a master.
- Deleting old data with delete-by-query instead of dropping time-based indices.

6 Key terms

Shard: a partition of an index; a full Lucene index

Refresh: makes buffered docs searchable (default 1 s)

Mapping: field types and analyzers: the index schema

Replica: copy of a shard on another node

Translog: per-shard write-ahead log for durability

ILM / ISM: automates rollover, tier moves and deletion

7 Interview questions

Q1. How do you size shards in Elasticsearch? BASIC

Aim for roughly 10–50 GB per shard, avoid too many shards per node, choose the primary count up front, and use time-based indices with rollover for data that keeps growing.

Q2. What do replicas give you? BASIC

Failover, since a replica is promoted when a primary's node dies, and more read throughput, since any copy can serve searches.

Q3. An index was created with 1 primary and is now 400 GB. What do you do? SCENARIO

Reindex into a new index with more primaries behind an alias (or use the _split API), then use rollover so it never grows unbounded.

Q4. Cluster is yellow after adding replicas. Why? SCENARIO

Some replicas can't be allocated, typically because there aren't enough nodes (a replica never shares a node with its primary) or disk watermarks are exceeded.

Q5. Bulk-loading 2 billion documents once. Which settings help? SCENARIO

Replicas 0 and refresh_interval -1 during the load plus bulk requests; restore both afterwards.

Q6. Why can't you just change number_of_shards on a live index? TRICKY

Documents are placed by hash(routing) % primaries, so a new count would move most documents; that needs a reindex, or split/shrink by exact factors.

Q7. A document was indexed successfully but a search right after doesn't find it. Bug? TRICKY

No: search is near real time, so it appears after the next refresh; GET by ID is real-time (lesson 20).

Next: SD-P-004 · Search Ranking backendroadmap.com